

Reinforcement Learning for Dynamic Vehicle Routing under Uncertain Traffic Conditions

Mini-Project Report

Master's Program of Excellence - Data Science and Information Systems Security
Sultan Moulay Slimane University - Multidisciplinary Faculty of Beni Mellal

Group: KHARBOUCH Essiddik, ER-RACHEDY Sana, BOUCHFIRA Hind, KHARBOUCH
Yahya

Supervisors: Pr. Hamza ALLAGA, Pr. Meriem EL HARKAOUI

Annee universitaire : 2025-2026

1. Introduction & Problem Statement

Vehicle routing is a core problem in transportation and logistics. Classical solvers assume travel times, demands, and the road network are known and fixed in advance, which breaks down under real traffic conditions: congestion can turn a route that was optimal at planning time into a poor one during execution. This project applies Reinforcement Learning to the Dynamic Vehicle Routing Problem (DVRP): a single capacitated vehicle must serve every customer on a graph at minimum cost, subject to per-customer delivery time windows, while traffic on each road segment is drawn from a spatially-correlated congestion field once per episode and is only observable within a limited radius of the vehicle's current position. The guiding question, taken directly from the project specification, is: how can Reinforcement Learning dynamically adapt vehicle routing decisions to uncertain traffic conditions while minimizing travel time, distance, and delays?

2. Problem Formulation (POMDP)

The environment (`src/env.py`, `DynamicCVRPEnv`) is a Gymnasium-compatible environment built around a complete graph over `{depot} + {customers}`. The real state - everything that exists in the environment, whether observed or not - includes every customer's position, demand, and time window, the vehicle's position and remaining capacity, and the true traffic multiplier on every road segment. The agent's observation differs from that real state in exactly one respect: traffic. Congestion is only revealed on edges within a fixed `observation_radius` of the vehicle's current position; everywhere else the observation reports zero (unobserved) with an explicit observed/unobserved mask. Because the agent must act on an incomplete view of the true state, this is formally a Partially Observable MDP (POMDP), not a plain MDP.

Nodes / graph size: 15 customers plus one fixed depot node ($N = 16$), positioned uniformly at random on a 1.0×1.0 square (the depot is fixed at its center). **Distance metric:** Manhattan (L_1) distance is used instead of Euclidean distance, since a vehicle on a grid-like street network cannot cut diagonally through a block; this also changes how much area a fixed observation radius reveals (a Manhattan disk of radius r covers $2 \times r^2$ versus $\pi \times r^2$ for a Euclidean disk of the same r), which was explicitly recalibrated and validated with a Monte Carlo check (see `notebook.md`, 2026-08-08).

Traffic / uncertainty: a spatially-correlated congestion field is generated from 2-4 random Gaussian "hotspots" per episode and mapped to a multiplier range $[1.0, 3.0]$ (free-flow to worst-case congestion). The multiplier for edge (i, j) is fixed for the whole episode but only revealed to the agent once the vehicle comes within `observation_radius = 0.3` of it - modelling the fact that a real driver only learns actual road congestion by approaching it, not by consulting a global traffic map. **Time windows:** enabled, each customer's delivery window is $[0, 120]$ wide 30-60 time units, following the standard VRPTW convention (Solomon 1987): arriving early triggers a free wait until the window opens (`service_start = max(arrival_time, window_start)`), only arriving after the window closes is penalized.

Action space

A fixed discrete action space of size N : choose the next node to visit. The set of actions that are actually legal changes at every step through action masking - a customer is masked out once served or if its demand exceeds the vehicle's remaining capacity, and the depot is masked out while the vehicle is already there (except to end the episode). Invalid actions are never sampled, in training or at evaluation.

Reward function

Component	Weight	When it applies
Travel cost penalty	$-1.0 \times (\text{distance} \times \text{traffic})$	Every move
Time-window violation	-10.0	Only if genuinely late (early arrival waits for free)
Delivery bonus	+1.0	Each successful delivery

Route completion bonus	+5.0	All customers served and vehicle back at depot
Unserved customer penalty	-20.0 per customer	Only if the episode is truncated (ran out of steps) with customers still unserved

3. Agent Architecture

Both learning agents share a graph-neural-network perception backbone (`src/gnn_encoder.py`): the dense per-step observation (node features, distance matrix, observed-traffic matrix, observation mask) is converted into a complete directed graph ($N \times (N-1)$ edges), and two stacked edge-conditioned graph convolution layers (PyTorch Geometric NNConv) propagate information two hops across it. For every edge, a small neural network maps its [distance, observed traffic, observed flag] features to a full weight matrix, so a message travelling along a calm, well-observed road is treated differently from one along a distant, congested, or still-unknown one. The encoder outputs per-node embeddings, which a linear head turns into one Q-value (GNN-DQN) or one policy logit plus a shared state value (GNN-PPO) per candidate next node; invalid actions are masked to a very large negative value before the arg-max / softmax in both agents.

Note on the GRU belief state. The original design considered a GRU carrying a recurrent "belief" across steps within an episode, to compensate for the traffic field only being partially observed. This was deliberately dropped after Week 2 (documented in `notebook.md`, 2026-08-15): a recurrent hidden state is only trained correctly on ordered, sequential transitions, but the DQN replay buffer samples randomly-ordered transitions from many different episodes, which breaks that sequential dependency without extra machinery (sequence-chunked replay, burn-in) that was out of scope for this timeline. Separately, the environment's observed-traffic mask never resets within an episode - it only grows as the vehicle moves - so every observation already carries a running summary of everything revealed so far. In effect, the belief the GRU was meant to build is already externalized into the environment's persistent state, which removes most of the marginal benefit a separate learned memory would add.

GNN-DQN (value-based, `src/dqn.py`): Double DQN - the online network selects the next state's action, the target network evaluates it, reducing the overestimation bias of vanilla DQN. A replay buffer stores transitions with terminated and truncated tracked separately, since only a true termination should zero the bootstrap target - a truncated episode (hit `max_steps`) is not a genuine terminal state.

GNN-PPO (policy-based, `src/ppo.py`): actor-critic PPO sharing the same GNN encoder, with the clipped surrogate objective, Generalized Advantage Estimation, and an entropy bonus for exploration; on-policy updates run every 512 collected steps.

Baselines (`src/baselines.py`): a Random Policy (uniform over valid actions) and a Greedy Nearest-Neighbor heuristic (always moves to the nearest unserved customer by raw distance, matching the cahier des charges' literal definition rather than a traffic-adjusted cost).

4. Training Setup

	GNN-DQN	GNN-PPO
Total environment steps	20,000	20,000
Learning rate	0.001	0.0003
Discount (gamma)	0.99	0.99
Batch size	64	64
Replay buffer size	50,000	-
Epsilon-greedy schedule	1.0 -> 0.05 over 10,000 steps	-
Target net sync every	500 steps	-
Rollout length	-	512 steps
Epochs per rollout	-	10

GAE lambda / clip epsilon	-	0.95 / 0.2
Entropy coefficient	-	0.01
Checkpoint every	5,000 steps	5,000 steps

Both agents train on the same fixed seed (42) and the same environment configuration, for a matched total-step budget, so any gap between them reflects the algorithms themselves rather than unequal training resources.

5. Training Convergence

Both agents are periodically checkpointed and evaluated on 5 held-out seeds during training. GNN-DQN's return rises steadily and plateaus by roughly step 15,000; GNN-PPO improves more slowly and was still trending downward in cost/lateness at the 20,000-step cutoff, i.e. not yet fully converged within this training budget (a limitation noted for the comparison below, not a finished result).

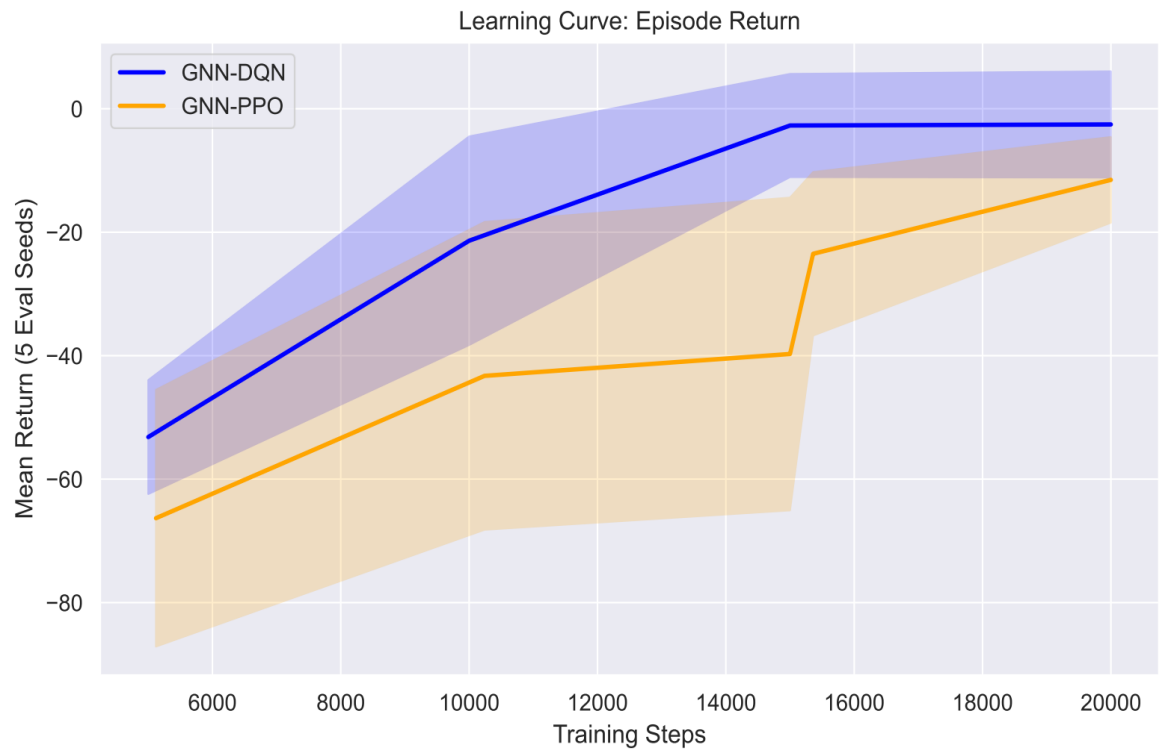


Figure 1 - Mean episode return vs. training steps, 5 held-out evaluation seeds, shaded band = +/-1 std.

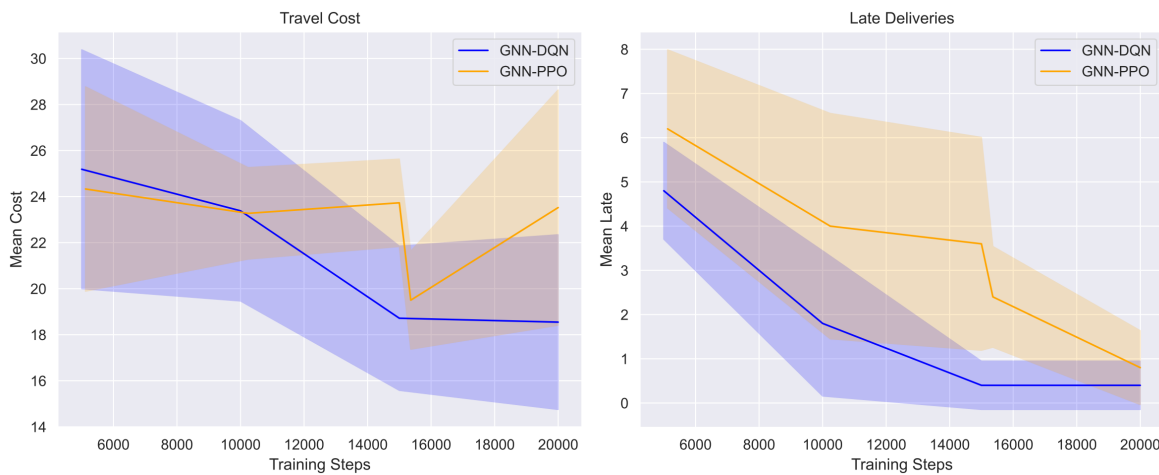


Figure 2 - Travel cost and late-delivery count vs. training steps for both learned agents.

6. Comparative Evaluation

All four routing methods (Random, Greedy Nearest-Neighbor, GNN-DQN, GNN-PPO) are evaluated on the same 100 held-out seeds (seeds 42-141), unseen during training. Final checkpoints (step 20,000) are used for both learned agents, and both act greedily (no exploration) at evaluation time.

Method	Return (mean+/-std)	Cost (mean+/-std)	On-time	Waited	Late
Random Policy	-72.27 +/- 22.61	22.77 +/- 5.14	5.2	2.8	7.0
Greedy Nearest-Neighbor	-57.71 +/- 21.51	13.01 +/- 2.59	5.4	3.1	6.5
GNN-DQN	-1.69 +/- 7.16	17.59 +/- 3.37	7.3	7.3	0.4
GNN-PPO	-19.91 +/- 18.11	21.21 +/- 4.87	7.9	5.2	1.9

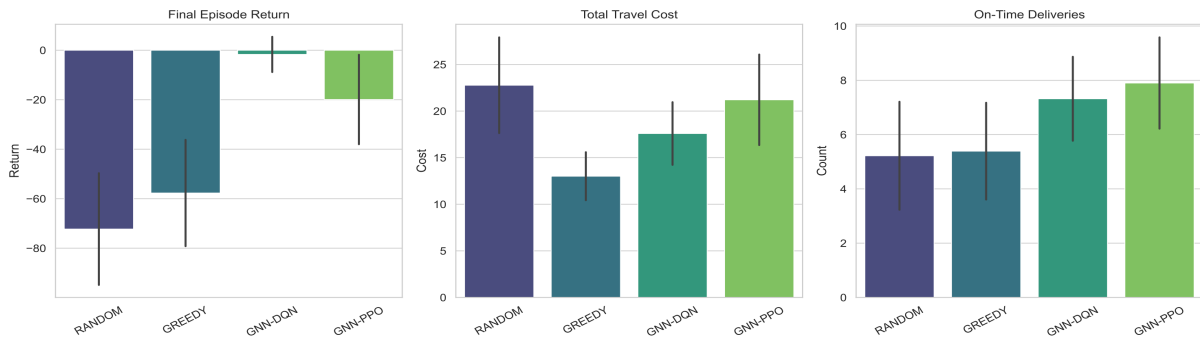


Figure 3 - Final episode return, total travel cost, and on-time delivery count, all four methods, 100 held-out episodes.

7. Discussion

GNN-DQN reaches a mean return of -1.69 versus Greedy Nearest-Neighbor's -57.71 - a 97% improvement - and Random's -72.27. The late-delivery rate falls from 43.1% for Greedy to 2.7% for GNN-DQN. Notably, this improvement is not achieved through a lower travel cost - Greedy actually stays cheaper on raw distance (13.01 vs. 17.59) - but through deliberately waiting more (7.3 waits/episode vs. Greedy's 3.1), exploiting the reward's asymmetry between a free early wait and a heavily penalized late arrival. This is the project's core empirical finding: a distance-minimizing heuristic cannot, by construction, fix time-window compliance - it has no notion of when to wait - while a learned agent conditioned on the traffic and time-window features can trade a small distance cost for a much larger compliance gain.

GNN-PPO shows the same qualitative pattern (return -19.91, late rate 12.5%) but a smaller effect than GNN-DQN. Given the learning curves in Section 5 show GNN-PPO still improving, not yet plateaued, at the shared 20,000-step training cutoff, this gap is best read as GNN-PPO being undertrained relative to GNN-DQN rather than a weaker method in principle - Double DQN's replay buffer reuses every transition many times over, giving it more effective gradient steps per environment step than PPO's strictly on-policy updates, which is a plausible mechanism for GNN-DQN converging faster within an equal environment-step budget. A fair, decisive comparison between the two algorithms would need either a larger step budget for GNN-PPO or a training-time budget (wall clock) matched instead of an environment-step budget.

8. Conclusion and Future Work

This project delivers a Gymnasium-based DVRP simulator with Manhattan-distance travel costs, a spatially-correlated and only locally-observable traffic field, and per-customer time windows; two graph-neural-network Deep RL agents (GNN-DQN and GNN-PPO) sharing an NNConv perception backbone with action masking; two heuristic baselines; and a common evaluation protocol across return, cost, and time-window compliance. The central finding - a learned agent trades a small increase

in travel distance for a large reduction in time-window violations by learning when to wait, something a distance-only heuristic cannot do - directly answers the problem statement in Section 1. Future work, out of scope for this first edition, includes multi-vehicle coordination, real-time traffic feeds (OSM/GPS), and training GNN-PPO for a larger or wall-clock-matched budget to settle the DQN-vs-PPO comparison decisively.